

МАТЕРИАЛИ ЗА ПОДГОТОВКА

Контролна работа по Обектно ориентирано програмиране

Клас: 12б | Първи срок на учебната 2025/2026 година

Информация за контролната работа:

- Брой варианти: 2
- Брой въпроси на вариант: 10
- Точки на въпрос: 10 точки (по 2 точки на всяко празно място)
- Общо точки: 100
- Формат: Попълване на празни места в C# код
- Време: 90 минути

1. КЛАСОВЕ И ОБЕКТИ

Дефиниране на класове

Класът е шаблон за създаване на обекти. В C# класът се дефинира с ключовата дума **class**, следвана от името на класа. Класът може да съдържа полета (fields), свойства (properties), методи (methods) и конструктори (constructors).

```
public class Student {  
    private string name; // private поле  
  
    public Student(string name) { // Конструктор  
        this.name = name;  
    }  
  
    public string Name { // Свойство  
        get { return name; }  
        set { name = value; }  
    }  
}
```

Модификатори за достъп

Модификаторите за достъп определят видимостта на членовете на класа:

Основни модификатори:

- **public** - достъпен навсякъде (извън класа, в наследниците)
- **private** - достъпен само в рамките на същия клас
- **protected** - достъпен в класа и в наследниците

Конструктори

Конструкторът е специален метод, който се извиква при създаване на обект. Името му съвпада с името на класа и няма тип на връщане. Използва се за инициализация на полетата на обекта.

```
public class Person {  
    private string name;  
  
    public Person(string name) { // Конструктор  
        this.name = name;  
    }  
}
```

Свойства (Properties)

Свойствата предоставят контролиран достъп до private полета. Те имат **get** и/или **set** аксесори. В **set** аксесора, стойността се достъпва чрез ключовата дума **value**.

```
public string Name {  
    get { return name; } // Четене на стойността  
    set { name = value; } // Задаване на стойността  
}
```

2. НАСЛЕДЯВАНЕ

Основна концепция

Наследяването позволява на един клас да наследи членовете на друг клас. Производният клас (наследник) получава достъп до **public** и **protected** членовете на базовия клас.

```
public class Vehicle { // Базов клас
    protected string brand; // protected - достъпен в наследниците

    public Vehicle(string brand) {
        this.brand = brand;
    }
}

class Car : Vehicle { // Производен клас
    public Car(string brand) : base(brand) { // Извикване на
        // конструктора на базовия клас
    }
}
```

virtual и override

Ключовата дума **virtual** се използва в базовия клас, за да позволи презаписване на метод в производния клас. Ключовата дума **override** се използва в производния клас за презаписване на virtual метода.

```
public class Animal {
    public virtual void MakeSound() { // virtual метод
        Console.WriteLine("Животно издава звук");
    }
}

class Dog : Animal {
    public override void MakeSound() { // override метод
        Console.WriteLine("Кучето лае");
    }
}
```

base ключова дума

Ключовата дума **base** се използва за достъп до членовете на базовия клас. Най-често се използва за извикване на конструктора на базовия клас: **base(параметри)**.

3. АБСТРАКТНИ КЛАСОВЕ

Дефиниране на абстрактен клас

Абстрактният клас не може да бъде инстанциран директно. Съдържа поне един абстрактен метод (метод без имплементация). Производните класове трябва да имплементират всички абстрактни методи.

```
public abstract class Animal {
    protected string name;

    public Animal(string name) {
        this.name = name;
    }

    public abstract void MakeSound(); // Абстрактен метод без
имплементация
}

class Dog : Animal {
    public Dog(string name) : base(name) {
    }

    public override void MakeSound() { // Имплементация на
абстрактния метод
        Console.WriteLine($"{name} лае");
    }
}
```

4. ИНТЕРФЕЙСИ

Дефиниране на интерфейс

Интерфейсът дефинира договор за поведение. Класът може да имплементира множество интерфейси. Всички методи в интерфейса са публични и абстрактни по подразбиране.

```
interface IShape {
    double CalculateArea();
    double CalculatePerimeter();
}

class Circle : IShape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double CalculateArea() {
        return Math.PI * radius * radius;
    }

    public double CalculatePerimeter() {
        return 2 * Math.PI * radius;
    }
}
```

5. СТАТИЧНИ ЧЛЕНОВЕ

Статични полета и методи

Статичните членове принадлежат на класа, а не на конкретен обект. Те се споделят между всички инстанции на класа. Статичните методи се извикват чрез името на класа, а не чрез обект.

```
class Counter {  
    private static int count = 0;  // Статично поле  
  
    public static void Increment() {  // Статичен метод  
        count++;  
    }  
  
    public static int GetCount() {  
        return count;  
    }  
}  
  
// Използване:  
Counter.Increment();  // Извикване на статичен метод  
int total = Counter.GetCount();
```


6. GENERICS

Generic класове

Generics позволяват създаването на класове и методи, които работят с различни типове данни. Типът се задава при създаване на обекта. Обикновено се използва буквата **T** за тип параметър.

```
class Box<T> {  
    private T item;  
  
    public Box(T item) {  
        this.item = item;  
    }  
  
    public T GetItem() {  
        return item;  
    }  
}  
  
// Използване:  
Box<string> stringBox = new Box<string>("Hello");  
Box<int> intBox = new Box<int>(42);
```

7. LINQ (Language Integrated Query)

Основни LINQ методи

Често използвани LINQ методи:

- **Where** - филтрира елементи по условие
- **OrderBy** - сортира във възходящ ред
- **OrderByDescending** - сортира в низходящ ред
- **Take** - взема първите N елемента
- **ToList** - конвертира в List
- **AsEnumerable** - конвертира в IEnumerable

```
List<Student> students = GetStudents();
var result = students
    .Where(s => s.Grade > 4.50)           // Филтриране
    .OrderByDescending(s => s.Grade)      // Сортиране
    .Take(5);                            // Взимане на първите 5

foreach (var student in result) {
    Console.WriteLine($"{student.Name}: {student.Grade}");
}
```

8. КОЛЕКЦИИ

List<T>

List<T> е динамичен масив, който може да расте и намалява. Основни методи:

- **Add(item)** - добавя елемент в края на списъка
- **Count** - свойство, което връща броя елементи
- **new List<T>()** - създаване на нов списък

```
List<int> numbers = new List<int>();  
numbers.Add(10);  
numbers.Add(20);  
int count = numbers.Count;
```

foreach цикъл

foreach цикълът се използва за итерация през колекции. Синтаксис: **foreach (тип променлива in колекция)**

```
foreach (var item in collection) {  
    Console.WriteLine(item);  
}
```

9. ОБРАБОТКА НА ИЗКЛЮЧЕНИЯ

try-catch-finally

Блокът **try** съдържа код, който може да хвърли изключение. Блокът **catch** прихваща и обработва изключенията. Блокът **finally** винаги се изпълнява, независимо дали е възникнало изключение.

```
try {
    int a = 10;
    int b = 0;
    int result = a / b; // Може да хвърли DivideByZeroException
} catch (DivideByZeroException ex) {
    Console.WriteLine($"Грешка: {ex.Message}");
} finally {
    Console.WriteLine("Операцията завърши"); // Винаги се изпълнява
}
```

int.Parse()

Методът **int.Parse()** конвертира низ в число. Ако низът не може да бъде конвертиран, хвърля **FormatException**.

```
try {
    string input = "123";
    int number = int.Parse(input); // Конвертира "123" в 123
} catch (FormatException ex) {
    Console.WriteLine($"Грешка при конвертиране: {ex.Message}");
}
```

10. ОПЕРАТОРИ И СИНТАКСИС

Аритметични оператори

- + - събиране
- - - изваждане
- * - умножение
- / - деление
- ++ - инкремент (увеличаване с 1)

new ключова дума

Ключовата дума **new** се използва за създаване на нов обект от даден клас. Синтаксис: **new ИмеНаКлас(параметри)**

```
Calculator calc = new Calculator(); // Създаване на нов обект
```

11. ПОЛЕЗНИ СЪВЕТИ ЗА КОНТРОЛНАТА РАБОТА

Как да се подготвите:

- Прегледайте всички ООП концепции: класове, наследяване, абстрактни класове, интерфейси
- Упражнете се с модификаторите за достъп: public, private, protected
- Запомнете разликата между virtual и override
- Научете синтаксиса на конструкторите и използването на base
- Упражнете се с LINQ методите: Where, OrderBy, OrderByDescending, Take
- Прегледайте работата със статични членове
- Упражнете се с generics и синтаксиса <T>
- Прегледайте обработката на изключения: try-catch-finally
- Упражнете се с колекции: List, foreach цикъл
- Внимавайте на синтаксиса: всяка скоба, точка и запетая са важни!

По време на контролната работа:

- Прочетете внимателно всеки въпрос и обяснението
- Погледнете целия код преди да започнете да попълвате
- Помнете, че всеки въпрос има точно 5 празни места
- Проверете дали отговорите ви са логически свързани с контекста
- Внимавайте на главни и малки букви в отговорите
- Проверете синтаксиса на кода след попълване

ПРОФЕСИОНАЛНА ГИМНАЗИЯ „НИКОЛА ЙОНКОВ ВАПЦАРОВ“

гр. Белица 2780, Обл. Благоевград, ул. "Димо Хаджидимов" 17

Тел: 0983605220 | e-mail: info-105002@edu.mon.bg